

Lambo: Shared Graph Memory for Multi-Agent Software Development

Narayan SS
nrn@lambo.dev

September 5, 2026

Abstract

Autonomous coding agents operate within ephemeral process lifecycles. When agents complete execution, their intermediate reasoning and architectural context vanish. Similarity retrieval alone does not explicitly encode dependency or invalidation relationships. We present Lambo, an in-memory topological memory daemon for multi-agent software engineering. Lambo models development state as a directed typed graph with an earned canonization lifecycle. Concurrent agents submit asynchronous writes through per-agent lanes with individual receipt states. Context retrieval uses three-leg hybrid candidate scoring and prioritized breadth-first topological expansion. Phase one merges lexical, recency, and dense vector signals. Phase two traverses structural dependency edges while respecting invalidation semantics. Phase three enforces canonical-first ranking and hot-list conflict preservation under token budgets. We report operational telemetry from two production rigs using nominal 5-minute health sampling. The extract contains 2,495 CUDA and 3,429 Metal heartbeat snapshots. At the nominal interval, those counts correspond to 8.7 and 11.9 interval-equivalent days within 12.7 and 16.0 day windows. Deduplication rates reached 12.0% during synchronized review swarms on Metal (against 0.8% in single-agent sessions) and 4.3% among swarm-named agents on CUDA (against 1.7% in non-swarm streams, 2.2% aggregate). The tracked infrastructure counters were zero in this extract, which does not establish zero failures or data loss. CUDA inspect misses were 22 of 82, while Metal had one miss in four calls and is too small for a cross-rig rate comparison. No concepts reached Canonical status, and the observed Metal store had no concepts eligible for the first promotion gate. We discuss citation dilution as a separate hypothesis and identify controlled retrieval evaluation as future work.

1 Introduction

Autonomous software engineering agents increasingly collaborate in swarms. Modern workflows deploy concurrent agents to refactor code, run tests, and remediate review findings. However, each agent invocation is fundamen-

tally ephemeral. When an agent session terminates, its internal reasoning context disappears. Subsequent agents must re-derive architectural decisions from scratch.

Maintaining coherent development context requires representing both decisions and their revisions. Flat markdown scratchpads and vector stores make different tradeoffs in representing relationships and supersession. Flat vector retrieval stores [1–3] commonly rank text chunks with similarity signals. This daemon explores explicit typed edges for codebase constraints and invalidation context. Graph retrieval frameworks [4, 5] emphasize graph-based retrieval over document collections. Lambo focuses on agent-written development state and local mutation handling.

To resolve these challenges, we present Lambo. Lambo is an in-memory topological memory daemon designed for multi-agent development. Lambo runs as a local supervised daemon exposing a Model Context Protocol (MCP) interface. Agents record typed architectural concepts and directed relationships during execution. Concepts represent logic invariants, structural constraints, and runtime observations.

Lambo uses per-agent admission lanes for asynchronous `derive` and `record_action` work. These calls receive composite receipt identifiers and may be awaited by identifier when an agent needs applied state. Retrieval operates via a three-phase hybrid topological pipeline. Phase one computes the maximum merge across keyword, recency, and dense vector candidates. Phase two performs prioritized breadth-first search across architectural dependency edges. Phase three applies daemon composite scoring, hot-list conflict preservation, and canonical-first ordering.

We report a systems experience across discrete GPU and unified-memory rigs. The nominal windows are 12.7 and 16.0 days, with 8.7 and 11.9 interval-equivalent days of snapshots. The extract contains zero values for selected counters, observed deduplication regimes, and model-family content-length differences that are confounded by task and model mix. These observations motivate improvements to retrieval evaluation, promotion eligibility, and tool interfaces.

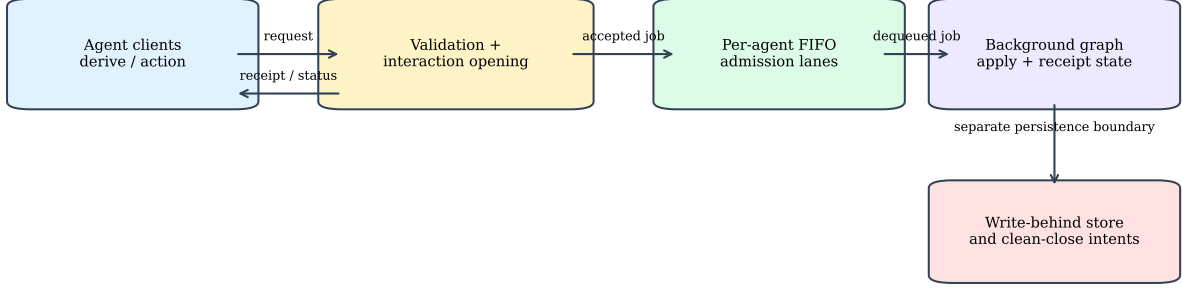


Figure 1: Lambo architecture as inspected at engine commit `e11fb06`. Qualifying asynchronous calls receive composite receipts. Applied receipt state and write-behind persistence are separate boundaries.

2 System Architecture and Invariants

2.1 Single-Writer Daemon Topology

Figure 1 illustrates the system architecture of Lambo. Lambo enforces a single-writer daemon topology. One supervised background daemon holds exclusive write authority over a durable session. Agents communicate with the daemon via Unix domain sockets or local HTTP streams. The design centralizes graph mutation in one daemon process while other callers may issue concurrent reads.

2.2 Asynchronous Write Queue and Receipts

`derive` and `record_action` may acknowledge after admission while embedding and insertion continue in the background. `reserve` remains synchronous because its result gates the caller’s next action. Each admitted job receives a composite receipt identifier containing an epoch, time, and sequence component. Receipt numbers are assigned before lane admission, so their numeric order is not a global drain or commit order. Each agent has a FIFO admission lane for sequential submissions, while lanes from different agents may run concurrently. A caller can query an individual receipt for an applied, failed, pending, or replay-related state. An applied receipt records ordinary graph-path completion. It does not itself assert that a disk flush has completed. The inspected implementation uses write-behind persistence and durable intents at clean close. This report therefore separates application state from the persistence boundary.

2.3 Level-B Storage Pluggability

Lambo implements Level-B storage pluggability via compile-time Cargo feature flags. The core graph engine accesses persistence through a shared storage interface. Supported engines include embedded SQLite, PostgreSQL, and CockroachDB. Adapters implement the same storage contract while retaining backend-specific

Algorithm 1 Asynchronous Ingestion and Receipt Scheduling

Require: Queue Q_{in} , In-Memory Graph $\mathcal{G}$, Persistent Store $\mathcal{S}$

Ensure: Per-job receipt state and per-agent FIFO admission

```

1: function SUBMITASYNC( $m$ , agent_id)
2:   Validate and open the interaction synchronously
3:    $r \leftarrow \text{CompositeReceipt}(\text{epoch}, \text{time}, \text{sequence})$ 
4:   Check admission bounds under graph and lane locks
5:   Refuse admission if the lane or queue is full
6:   Record an intent in the write-behind log and enqueue  $\text{Job}(r, m)$ 
7:   return  $r$ 
8: end function

9: function AGENTLANEWORKER(agent_id)
10:  while DaemonIsRunning() do
11:    Dequeue the next job from this agent’s lane
12:    Embed and apply the job through the ordinary graph path
13:    Set its receipt to APPLIED or FAILED
14:    Write-behind persistence flushes separately from receipt application
15:  end while
16: end function

```

transaction and vector-search implementations. The deployment evaluation reported here uses SQLite on both rigs.

2.4 The Embedding Contract Invariant

Vector spaces are fragile under encoder model migrations. Comparing vector embeddings generated by divergent models corrupts metric spaces. Lambo enforces an immutable session-level embedding contract. The contract is defined by a three-tuple:

$$\mathcal{C} = (\kappa, \mu, d) \quad (1)$$

Here, κ identifies the provider, μ defines the model name, and d specifies the exact vector dimensionality. The storage engine records $\mathcal{C}$ at session initialization. Any subsequent process that attaches with a mismatched contract is rejected immediately. The check prevents mixing incompatible declared embedding configurations within a session.

3 Formal Memory Model

3.1 The Directed Attributed Multi-Graph

Lambo models development state as a directed, attributed multi-graph:

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}) \quad (2)$$

The node set $\mathcal{V}$ represents typed concepts. Each concept node $c \in \mathcal{V}$ possesses a type classification:

$$\tau(c) \in \{\text{Logic, Constraint, Observation, Entity, Resource}\} \quad (3)$$

Each node maintains a text payload, creation timestamp t_c , access counter α_c , and dense embedding $\mathbf{e}_c \in \mathbb{R}^d$.

Edges in $\mathcal{E}$ represent typed architectural dependencies:

$$\epsilon(e) \in \{\text{Dependency, Causal, Hierarchical, CoOccurrence, Semantic}\} \quad (4)$$

In addition, bipartite provenance edges track agent actions. These include **Derives** edges from interactions to concepts and **Temporal** ordering edges. Invalidation is represented by edge polarity rather than a separate **Invalidates** edge type.

3.2 Three-Leg Hybrid Candidate Retrieval

Given a natural language query q , the recall engine retrieves candidate nodes from three orthogonal legs. First, the lexical leg evaluates term frequencies over an inverted text index using BM25 [6].

$$S_{\text{lex}}(q, c) = \text{BM25}(q, c) \quad (5)$$

To prevent lexical candidates from being evicted by flat scores, the lexical search over-fetches with multiplier $M = 4$.

Second, the recency leg retrieves concepts derived within the $N_{\text{recent}} = 3$ most recent interactions. Lambo assigns these candidates a fixed recency score. Members of this leg receive a flat floor score:

$$S_{\text{rec}}(c) = \rho_{\text{floor}} = 0.35 \quad (6)$$

An internal calibration using BGE-M3 [7] contained a 0.3991 true-recall score. The [calibration artifact](#) documents this fixture-specific observation and the floor adjustment. Setting $\rho_{\text{floor}} = 0.35$ is an implementation heuristic whose interaction with raw BM25 and cosine scales needs evaluation.

Third, the vector leg projects query text into vector $\mathbf{e}_q \in \mathbb{R}^d$ and computes cosine similarity.

$$S_{\text{vec}}(q, c) = \cos(\mathbf{e}_q, \mathbf{e}_c) = \frac{\mathbf{e}_q \cdot \mathbf{e}_c}{\|\mathbf{e}_q\|_2 \|\mathbf{e}_c\|_2} \quad (7)$$

Phase one merges candidate scores using a maximum selection rule.

$$S_1(q, c) = \max(S_{\text{lex}}(q, c), S_{\text{vec}}(q, c), S_{\text{rec}}(c)) \quad (8)$$

The implementation returns the over-fetched union without Phase-1 top- K truncation. Final selection occurs downstream during Phase-3 assembly. Because BM25, cosine similarity, and the recency floor are max-merged without normalization, this is a heuristic candidate score whose cross-leg scale effects require controlled ablation.

3.3 Phase-2 Prioritized Breadth-First Expansion

The candidate set forms the frontier for Phase-2 graph traversal. Unlike Personalized PageRank in HippoRAG [5, 8], Lambo prioritizes directed architectural edges. Starting from Phase-1 seeds at depth zero, Lambo executes a breadth-first search up to depth $D_{\text{max}} = 2$. At each frontier node, traversal follows edges according to strict type priority:

$$\begin{aligned} \text{Dependency} &\succ \text{Causal} \succ \text{Hierarchical} \\ &\succ \text{CoOccurrence} \succ \text{Semantic} \end{aligned} \quad (9)$$

Provenance edges are excluded from Phase-2 expansion. A visited set cycle guard ensures that each concept is enqueued at most once. The earliest discovery path defines node membership.

Furthermore, Lambo applies a transitive closure over chunk grouping identifiers. Concepts sharing a `chunk_group_id` with an expanded node are force-included as siblings. Phase-1 candidates retain their S_1 scores, whereas newly expanded neighbors enter Phase 3 with an initial score of zero.

3.4 Phase-3 Composite Scoring and Token Assembly

Phase three computes final composite scores for all expanded concepts:

$$S_{\text{final}}(c) = w_{\text{daemon}} \cdot S_{\text{daemon}}(c) + w_{\text{query}} \cdot S_1(q, c) \quad (10)$$

In the inspected default configuration, $w_{\text{daemon}} = 0.5$ and $w_{\text{query}} = 0.5$. The frozen telemetry does not retain a complete configuration record. The daemon score $S_{\text{daemon}}(c) \in [0, 1 + B_{\text{max}}]$ evaluates session-relative topological health:

$$\begin{aligned} S_{\text{daemon}}(c) &= 0.25 \cdot R(c) + 0.20 \cdot F(c) + 0.20 \cdot A(c) \\ &\quad + 0.35 \cdot \Delta(c) + \beta_{\text{edge}}(c) + \mu_{\text{type}}(\tau(c)) \end{aligned} \quad (11)$$

The normalized component metrics are defined as follows:

- **Recency:** $R(c) = \frac{t_{\text{touch}}(c) - t_{\text{start}}}{t_{\text{end}} - t_{\text{start}}}$, normalized over session duration.
- **Frequency:** $F(c) = \min(1.0, \frac{\alpha_c}{10.0})$, saturating at ten accesses.
- **Session Activity:** $A(c) = \frac{|\{i \in \mathcal{I} \mid i \xrightarrow{\text{Derives}} c\}|}{|\mathcal{I}|}$.
- **Graph Density:** $\Delta(c) = \frac{\deg(c)}{\max_{u \in \mathcal{V}} \deg(u)}$, scaled by maximum degree.
- **Edge Bonus:** $\beta_{\text{edge}}(c) = \min(0.25, \sum_{e \sim c} b(\epsilon(e)))$.
- **Type Modifier:** $\mu_{\text{type}}(\text{Constraint}) = +0.15$, $\mu_{\text{type}}(\text{Entity}) = +0.05$, $\mu_{\text{type}}(\text{Logic}) = +0.05$, $\mu_{\text{type}}(\text{Resource}) = 0.0$, and $\mu_{\text{type}}(\text{Observation}) = -0.10$.

Each weighted component is clamped to $[0, 1]$, and the composite is clamped to $[0, 1.4]$ under these default weights. A zero-duration recency extent yields one. Empty interaction or degree denominators yield zero. The Metal deployment report records zero access counters, so the frequency term supplied no recall-usage signal in that store.

3.5 Canonical-First Ordering and Token Budgets

Hits are sorted by canonical priority and final score:

$$\text{RankKey}(c) = (\mathbb{I}_{\text{canonical}}(c), S_{\text{final}}(c), -\text{NodeId}(c)) \quad (12)$$

Here, $\mathbb{I}_{\text{canonical}}(c) \in \{0, 1\}$ guarantees that Canonical concepts appear ahead of non-canonical concepts. Active conflict warnings are rendered separately from budgeted context blocks. A block dropped by the token budget can still contribute a warning. Text context is assembled from a score-ordered prefix of whole concept blocks that fits within token budget T_{max} . Algorithm 2 formalizes this complete retrieval workflow.

4 The Promotion Engine and Lifecycle

4.1 The Three-Stage Canonization Pipeline

The daemon executes periodic background canonization evaluations. Concepts transition across three formal lifecycle states:

$$\text{Candidate} \xrightarrow{\text{Stage 1, 2}} \text{Venerable} \xrightarrow{\text{Stage 3}} \text{Canonical} \quad (13)$$

While Voyager [9] accumulates executable skills and Reflexion [10] relies on verbal reflection, Lambo enforces earned canonization through topological gates. Promotion requires passing four orthogonal evidence gates. Stage one evaluates the garbage collection survival count.

$$\text{gc_survived}(c) \geq 3 \quad (14)$$

Algorithm 2 Three-Phase Topological Recall (Simplified)

Require: Query string q , Graph $\mathcal{G}$, Inverted Index $\mathcal{I}$, Limit K , Depth D_{max}

Ensure: Budgeted context and separately rendered warnings

```

1:  $\mathcal{C}_{\text{lex}} \leftarrow \mathcal{I}.\text{Search}(q, K \times 4)$ 
2:  $\mathcal{C}_{\text{rec}} \leftarrow \{c \in \mathcal{V} \mid c \text{ from 3 most recent interactions}\}$ 
3:  $\mathbf{e}_q \leftarrow \text{Embedder.Embed}(q)$ 
4:  $\mathcal{C}_{\text{vec}} \leftarrow \text{VectorSearch}(\mathbf{e}_q, K)$ 
5:  $\mathcal{C}_1 \leftarrow \emptyset$ 
6: for  $c \in \mathcal{C}_{\text{lex}} \cup \mathcal{C}_{\text{rec}} \cup \mathcal{C}_{\text{vec}}$  do
7:    $s \leftarrow \max(S_{\text{lex}}(q, c), S_{\text{vec}}(q, c), S_{\text{rec}}(c))$ 
8:    $\mathcal{C}_1 \leftarrow \mathcal{C}_1 \cup \{(c, s)\}$ 
9: end for
10:  $\mathcal{S}_{\text{seeds}} \leftarrow \text{RankedUnion}(\mathcal{C}_1)$  ▷ unreduced candidate over-approximation
11:  $\mathcal{V}_{\text{bfs}} \leftarrow \mathcal{S}_{\text{seeds}}$ 
12:  $\mathcal{Q} \leftarrow \text{Queue}((c, 0) \mid c \in \mathcal{S}_{\text{seeds}})$ 
13:  $\text{Visited} \leftarrow \text{Set}(\mathcal{S}_{\text{seeds}})$ 
14: while  $\mathcal{Q} \neq \emptyset$  do
15:    $(u, d) \leftarrow \mathcal{Q}.\text{Pop}()$ 
16:   if  $d \geq D_{\text{max}}$  then
17:     continue
18:   end if
19:   for  $e \in \text{SortedEdges}(u.\text{OutEdges})$  do
20:      $v \leftarrow e.\text{Target}$ 
21:     if  $v \notin \text{Visited}$  and  $\epsilon(e) \in \text{TraversableTypes}$  then
22:        $\text{Visited.Add}(v)$ 
23:        $\mathcal{V}_{\text{bfs}} \leftarrow \mathcal{V}_{\text{bfs}} \cup \{(v, 0.0)\}$ 
24:        $\mathcal{Q}.\text{Push}((v, d + 1))$ 
25:     end if
26:   end for
27: end while
28:  $\mathcal{V}_{\text{all}} \leftarrow \mathcal{V}_{\text{bfs}} \cup \text{ChunkGroupSiblings}(\mathcal{V}_{\text{bfs}})$ 
29: for  $c \in \mathcal{V}_{\text{all}}$  do
30:    $S_{\text{final}}(c) \leftarrow 0.5 \cdot S_{\text{daemon}}(c) + 0.5 \cdot S_1(q, c)$ 
31: end for
32:  $\mathcal{H}_{\text{sorted}} \leftarrow \text{SortByCanonicalAndScore}(\mathcal{V}_{\text{all}})$ 
33:  $\mathcal{H} \leftarrow \text{TopKPlusRevalidatedHotMembers}(\mathcal{H}_{\text{sorted}}, K)$ 
34:  $\mathcal{W} \leftarrow \text{Warnings}(\mathcal{H})$ 
35:  $\text{context} \leftarrow \text{WholeBlockPrefix}(\mathcal{H}, T_{\text{max}})$ 
36: return (context,  $\mathcal{W}$ )
```

Stage two checks for derivation across distinct interactions.

$$\text{distinct_interactions}(c) \geq 3 \quad (15)$$

Stage two also requires structural edge coverage over session time.

$$\text{coverage}(c) = \frac{\max(t) - \min(t)}{t_{\text{extent}}} \geq 0.30 \quad (16)$$

Stage three requires a verified blast radius strictly exceeding five.

$$\text{blast_radius}(c) > 5 \quad (17)$$

4.2 Consensus Promotion Policies

Lambo supports two alternative promotion scorers:

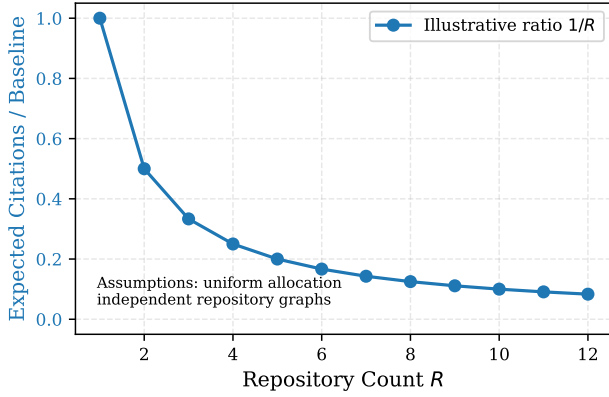


Figure 2: Illustrative inverse- R citation-density model. It assumes uniform allocation across independent repository graphs and is not a fitted deployment curve.

- **Solo Policy:** Focuses on longitudinal recurrence within single-developer workflows. It evaluates time-separated session counts without checking peer agent consensus.
- **Swarm Policy:** Enforces multi-agent consensus. Promotion requires concurrent corroboration across independent agent identities.

4.3 The Multi-Repository Dispersion Formulation

The [Metal operational report](#) records approximately 15,000 canonization cycles with zero Swarm-policy Canonical concepts. It also records `gc_survived=0` for every concept, so the first gate, `gc_survived ≥ 3`, already blocked promotion. This count cannot be attributed to repository dispersion. It is a workload and gate-state observation, not evidence that promotion permanently freezes or that namespace scoping supplies sufficient corroboration. We use the term *Multi-Repository Dispersion Hypothesis* for the following simplified model.

Let M denote the total interaction volume generated by an agent swarm. Assume an operator works across R independent repositories. The total concept space partitions into R disconnected subgraphs:

$$\mathcal{G} = \bigcup_{r=1}^R \mathcal{G}_r, \quad \mathcal{G}_i \cap \mathcal{G}_j = \emptyset \quad \forall i \neq j \quad (18)$$

Assuming uniform interaction distribution across repositories, the expected interactions per repository equals:

$$\mathbb{E}[M_r] = \frac{M}{R} \quad (19)$$

Let $\delta_r(c)$ denote the citation count for concept c over a fixed observation window. Assume the probability of citing c per interaction within its repository stays constant

as R changes. Under these assumptions, the expected count scales inversely with repository count:

$$\mathbb{E}[\delta_r(c)] = \frac{\delta_{\text{base}}(c)}{R} \quad (20)$$

Figure 2 depicts this relationship. For a fixed baseline citation density, the model reduces the expected local density by a factor of R . Citations may be diluted across disjoint problem domains under these assumptions. The derivation does not model task distributions, cross-repository concepts, gate-level eligibility, or agent behavior. It therefore motivates a future evaluation of promotion gates and namespace policies rather than prescribing a universal policy.

5 Empirical Evaluation

5.1 Experimental Setup

We observed Lambo across two production environments. The Metal window spans changing binaries and embedders, with per-session studio processes preceding the August 23 transition to supervised HTTP. The architecture description follows the inspected Metal revision [e11fb06](#), deployed September 1. Earlier observations include different configurations.

- **CUDA Rig:** AMD Ryzen 5 3600 CPU (6 cores, 12 threads) and 78 GB DDR4 RAM. It includes an NVIDIA GeForce RTX 4070 SUPER GPU (12 GB VRAM) running CachyOS Linux kernel 7.2.2. The daemon runs as a systemd unit using candle-cuda with BGE-M3 (1024 dimensions).
- **Metal Rig:** Apple MacBook Pro with M3 Pro processor (12 CPU cores, 18 GPU cores), 18 GB Unified Memory, and macOS 26. The daemon runs under launchd using the candle-metal backend.

5.2 Telemetry and Infrastructure Reliability

Table 1 summarizes automated telemetry over the multi-rig evaluation. The frozen extracts contain 2,495 CUDA and 3,429 Metal heartbeat snapshots. At the nominal five-minute interval, those are 8.7 and 11.9 interval-equivalent days, whereas the monitoring windows are 12.7 and 16.0 days. They do not measure uptime or continuous coverage. Every listed infrastructure counter is zero in these samples. These counters do not demonstrate zero daemon failures, restarts, or data loss outside their coverage. The 52 CUDA and 12 Metal process restarts require separate causal and recovery evidence.

In contrast to infrastructure health, tool execution telemetry revealed application-level failures. Across 1,020 tool calls on the CUDA rig, 25 calls returned errors. Twenty-two errors were `lambo_inspect` misses on approximate strings, and three were invalid reservation arguments. On the Metal rig, four inspect calls produced one

Table 1: Operational telemetry. Nominal windows and interval-equivalent snapshot counts are reported separately. Zero counters are observations in the frozen extracts.

Reliability Metric	CUDA Rig	Metal Rig
Monitoring Window	12.7 days	16.0 days
Heartbeat Snapshots (300 s)	2,495	3,429
Snapshot Interval Equivalents	8.7 days	11.9 days
Total Concepts in Store	1,819	889
Total Directed Edges	4,552	2,198
Daemon Process Restarts	52	12
<i>Infrastructure Health Counters</i>		
Dropped Ledger Lines	0	0
Write Queue Buffer Drops	0	0
Dead-Lettered Mutations	0	0
Unresolved Replay Debt	0	0
Degraded Episode Count	0	0
<i>MCP Application Tool Execution</i>		
Logged Tool Invocations	1,020	365
Overall Tool Error Rate	2.45% (25 / 1,020)	0.27% (1 / 365)
inspect Miss Rate	26.8% (22 / 82)	25.0% (1 / 4)
reserve Error Rate	10.3% (3 / 29)	N/A (0 calls)
recall / derive Errors	0.0% (0 / 797)	0.0% (0 / 346)

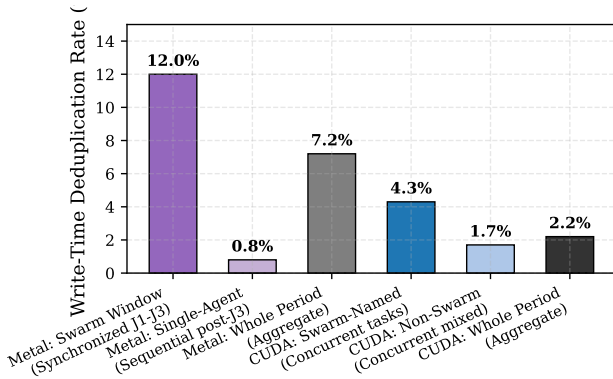


Figure 3: Observed write-time match rates across deployment regimes. Workload overlap, model mix, and deployment changes were not independently controlled.

miss. This denominator is too small to estimate a stable cross-rig rate. The CUDA misses are consistent with an affordance gap, but the dataset does not establish its cause. The `lambo_recall` tool renders text content without unique node identifiers. Approximate-string inspect calls suggest that explicit handles could improve tool use.

5.3 Deduplication Dynamics: Swarms vs. Streams

Lambo evaluates semantic deduplication during ingestion. In the inspected default configuration, an incoming concept is eligible for semantic matching at cosine similarity ≥ 0.85 . The frozen telemetry does not preserve all historical configuration overrides. Figure 3 and Table 2 detail observed match rates across operational regimes.

On the Metal rig, work followed a sharp temporal shift. During the multi-agent review swarm, multiple agents audited identical code changes simultaneously. In this over-

Table 2: Write-time deduplication across deployment regimes.

Deployment Regime	Created	Matched	Rate
<i>Metal Rig (Temporal Regime Shift)</i>			
Review Swarm (Aug 19–23)	456	62	12.0%
Single Stream (Aug 24–Sep 4)	376	3	0.8%
Metal Rig Aggregate	832	65	7.2%
<i>CUDA Rig (Continuous Concurrency)</i>			
Swarm-Named Agent IDs	309	14	4.3%
Non-Swarm Agent IDs	1,285	22	1.7%
CUDA Rig Aggregate	1,594	36	2.2%

Table 3: Concept content length distributions across both stores.

Model Group	<i>n</i>	Mean	p50	p90	>500B
<i>CUDA Rig Store (1,819 concepts)</i>					
Claude family	620	325 B	319 B	706 B	30.0%
GPT family	319	152 B	67 B	433 B	7.5%
Grok family	204	144 B	130 B	302 B	1.0%
CUDA Rig Aggregate	1,819	232 B	90 B	585 B	15.7%
<i>Metal Rig Store (889 concepts)</i>					
Claude family	713	407 B	356 B	918 B	38.7%
GPT family	43	196 B	164 B	395 B	4.7%
Grok family	29	62 B	44 B	135 B	0.0%
Metal Rig Aggregate	889	392 B	315 B	932 B	35.9%

lapping regime, deduplication reached 12.0%. When development transitioned to a single operator, deduplication dropped to 0.8%. On the CUDA rig, development operated under continuous concurrency with five to 23 active agents daily. Swarm-named task agents matched at 4.3%, whereas non-swarm agents matched at 1.7%. This telemetry is consistent with the hypothesis that write-time deduplication correlates with concurrent task overlap rather than store consolidation.

5.4 Model Verbal Drift and Concept Lengths

Table 3 presents concept payload lengths across frontier language model families. Both stores show differences in payload length among the recorded model-family groups. Across both platforms, the Claude family generated verbose multi-sentence paragraphs. On CUDA, Claude median concept length was 319 bytes, with 30.0% of concepts exceeding 500 bytes. On Metal, Claude concepts averaged 407 bytes, with 38.7% exceeding 500 bytes. Reasoning concepts for Logic and Constraint reached mean lengths of 703 bytes and 707 bytes. Conversely, GPT and Grok agents generated concise atomic statements. On CUDA, GPT median length was 67 bytes, with only 7.5% exceeding 500 bytes. Grok concepts averaged 144 bytes on CUDA and 62 bytes on Metal.

The family labels are confounded with task mix, prompting, and deployment conditions. The telemetry does not identify a causal effect of length on false positives or show that graph topology mitigates one.

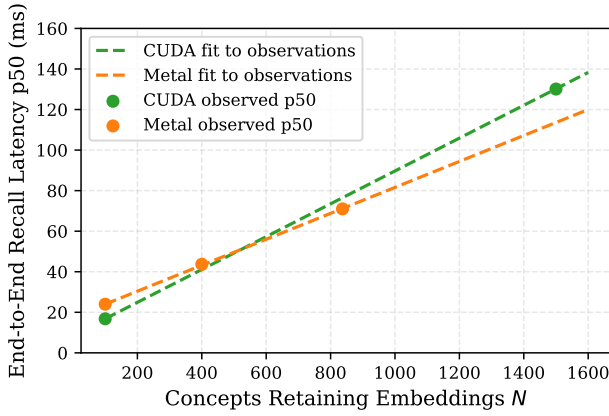


Figure 4: End-to-end recall p50 observations from Issue 8 and linear fits. Each CUDA size has 12 timed recalls after two warmups. Each Metal size has 24 timed recalls after two warmups. Dashed lines extend the fitted models beyond the measured sizes.

5.5 Coordination Telemetry: Advisory Locks vs. Receipts

Early agent memory designs proposed distributed advisory locks via `lambo_reserve`. Under that proposal, agents acquire temporary reservations before editing shared code. Our production telemetry demonstrated that agents rarely acquired advisory locks. Across 247 writes on the Metal rig, zero calls were made to `lambo_reserve`. On the CUDA rig, only 27 of 422 writes (6.4%) utilized soft locks. Furthermore, 22 of those 27 calls originated from a single agent. Only five of 63 distinct agents ever invoked reservations. Discoverability and identifier fragmentation are possible explanations for this low adoption. Ephemeral agent identifiers make lease coordination difficult to maintain across sessions. Instead, agents used asynchronous receipt-bearing writes. Issue 8 reports a 0.1 ms median CUDA derive acknowledgement. The Metal window also contains earlier synchronous calls. This allows agents to proceed without synchronous blocking.

5.6 Preliminary Operational Comparison

The [Issue 17 deployment report](#) describes Lambo answering four of five working questions in a comparison against Claude Code file memory. The published artifacts omit the query set, prompts, model version, token budget, baseline construction, and grading protocol. This anecdotal result motivates a reproducible comparison.

6 Systems Overheads and Hardware Analysis

6.1 The Vector Deserialization Bottleneck

Figure 4 reports end-to-end recall p50 rather than an isolated Phase-1 vector scan. CUDA observations were 16.8

ms at 100 concepts and 130.1 ms at 1,500 concepts. Metal observations were 24.0 ms at 100, 43.7 ms at 400, and 71.0 ms at 837 concepts. The [Issue 8 probe report](#) describes isolated store copies with embeddings retained on only n concepts. CUDA used 12 timed recalls per size after two warmups. Metal used 24 timed recalls per size, twelve distinct queries, and two warmups. The following linear fits summarize those observations:

$$T_{\text{CUDA}}(n) = 8.7 \text{ ms} + 81 \mu\text{s} \times n \quad (21)$$

$$T_{\text{Metal}}(n) = 17.6 \text{ ms} + 64 \mu\text{s} \times n \quad (22)$$

Profiling identified the primary bottleneck in text deserialization. The initial SQLite storage schema serialized dense vectors as decimal JSON arrays. At 1,500 concepts, a CUDA recall query parsed approximately 19.1 MB of decimal strings. This triggered 1,540,000 invocations of floating-point string parsers per query. The report attributes approximately 87% of recall time to the scan using the size-sweep slope and fixed-cost estimate.

Packed binary BLOB storage is a proposed optimization. The 20 to 30 ms range is a design target and has not been measured, so the report makes no slope, floor, or achieved-latency claim for it.

6.2 Unified Memory Operating System Compression

On the Apple Silicon Metal platform, we discovered a hardware-specific latency degradation. When the Lambo daemon remained idle for extended intervals, macOS compressed idle process pages. Out of a 1,429 MB process footprint, the memory manager compressed 1,417 MB. The process-level compression observation does not identify which individual allocations were compressed.

In the recorded idle-duration bins, recall latency varied as follows:

- **<2 minutes idle:** Recall latency p50 was 186 ms.
- **10 to 30 minutes idle:** Recall latency p50 rose to 567 ms.
- **>30 minutes idle:** Recall latency p50 reached 780 ms, with p90 reaching 9,059 ms.

These bins contain 19, 9, and 17 observations respectively and do not establish a causal operating-system mechanism. Periodic lightweight touches are a proposed mitigation that needs an intervention study.

7 Related Work

LLM Agent Memory. Seminal architectures established linear episodic memory streams [11]. Reflexion and Voyager explored iterative skill accumulation and verbal reinforcement learning [9, 10]. MemoryBank introduced forgetting curves inspired by psychological theories [2]. These systems explore different memory representations

and update policies. Lambo contributes a typed graph and promotion gates for a local multi-agent daemon.

Graph Retrieval Architectures. GraphRAG and HippoRAG use graph representations for retrieval [4, 5]. HippoRAG uses Personalized PageRank and GraphRAG builds graph summaries over corpora. Lambo targets typed development constraints updated during agent execution.

Agent Operating Systems and Shared Memory. MemGPT explored hierarchical memory paging for extended contexts [12]. A-MEM dynamically organizes linked memories, while Collaborative Memory addresses multi-user sharing and access control [13, 14]. MemClaw formulated governed shared memory for agent fleets [15]. Lambo contributes a local single-writer daemon architecture with per-agent FIFO admission lanes for qualifying asynchronous operations.

8 Conclusion

This systems experience documents a typed graph daemon, receipt-bearing asynchronous operations, and frozen operational telemetry. The extracts contain 5,924 heartbeat snapshots and zero values for selected counters, but do not prove absence of failures or data loss. Write-time deduplication was 12.0% in one Metal review-swarm window and 4.3% for CUDA swarm-named agent IDs. The inverse- R model is a hypothesis motivating controlled study of promotion gates and namespace policies. Future evaluation should hold memories, reader model, prompts, and token budgets constant, compare flat and graph baselines, and ablate graph expansion and invalidation handling.

References

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [2] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang, “Memorybank: Enhancing large language models with long-term memory,” *arXiv preprint arXiv:2305.10250*, 2023.
- [3] Z. Zhang, X. Bo, C. Ma, R. Li, X. Chen, Q. Dai, J. Zhu, Z. Dong, and J.-R. Wen, “A survey on the memory mechanism of large language model based agents,” *arXiv preprint arXiv:2404.13501*, 2024.
- [4] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, and J. Larson, “From local to global: A graph rag approach to query-focused summarization,” *arXiv preprint arXiv:2404.16130*, 2024.
- [5] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, “Hipporag: Neurobiologically inspired long-term memory for large language models,” *arXiv preprint arXiv:2405.14831*, 2024.
- [6] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: Bm25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [7] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, “M3-embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation,” *arXiv preprint arXiv:2402.03216*, 2024.
- [8] B. J. Gutiérrez, Y. Shu, W. Qi, S. Zhou, and Y. Su, “From rag to memory: Non-parametric continual learning for large language models,” in *Proceedings of the 42nd International Conference on Machine Learning*, 2025, pp. 21 497–21 515.
- [9] G. Wang, Y. Xie, Y. Jiang, A. Mandlkar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, “Voyager: An open-ended embodied agent with large language models,” *arXiv preprint arXiv:2305.16291*, 2023.
- [10] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” *arXiv preprint arXiv:2303.11366*, 2023.
- [11] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” *arXiv preprint arXiv:2304.03442*, 2023.
- [12] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, “Memgpt: Towards llms as operating systems,” *arXiv preprint arXiv:2310.08560*, 2023.
- [13] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, “A-mem: Agentic memory for llm agents,” *arXiv preprint arXiv:2502.12110*, 2025.
- [14] A. Rezazadeh, Z. Li, A. Lou, Y. Zhao, W. Wei, and Y. Bao, “Collaborative memory: Multi-user memory sharing in llm agents with dynamic access control,” *arXiv preprint arXiv:2505.18279*, 2025.
- [15] Y. Margalit, N. Cohen-Inger, E. Avram, R. Taig, and O. Margalit, “Governed shared memory for multi-agent llm systems,” *arXiv preprint arXiv:2606.24535*, 2026.